Solution: Valid Parentheses

Problem Statement

Determine if an input string containing only the characters '(', ')', '{', '}', '[', and ']' is valid. A string is considered valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Each close bracket has a corresponding open bracket of the same type.

Examples

1. - **Input**: `"(]"`
   - **Expected Output**: `false`
   - **Justification**: The opening parenthesis '(' is not closed by its corresponding closing parenthesis.
2. - **Input**: `"{[]}"`
   - **Expected Output**: `true`
   - **Justification**: The string contains pairs of opening and closing brackets in the correct order.
3. - **Input**: `"[{]}"`
   - **Expected Output**: `false`
   - **Justification**: The opening square bracket '[' is closed by a curly brace '}', which is incorrect.

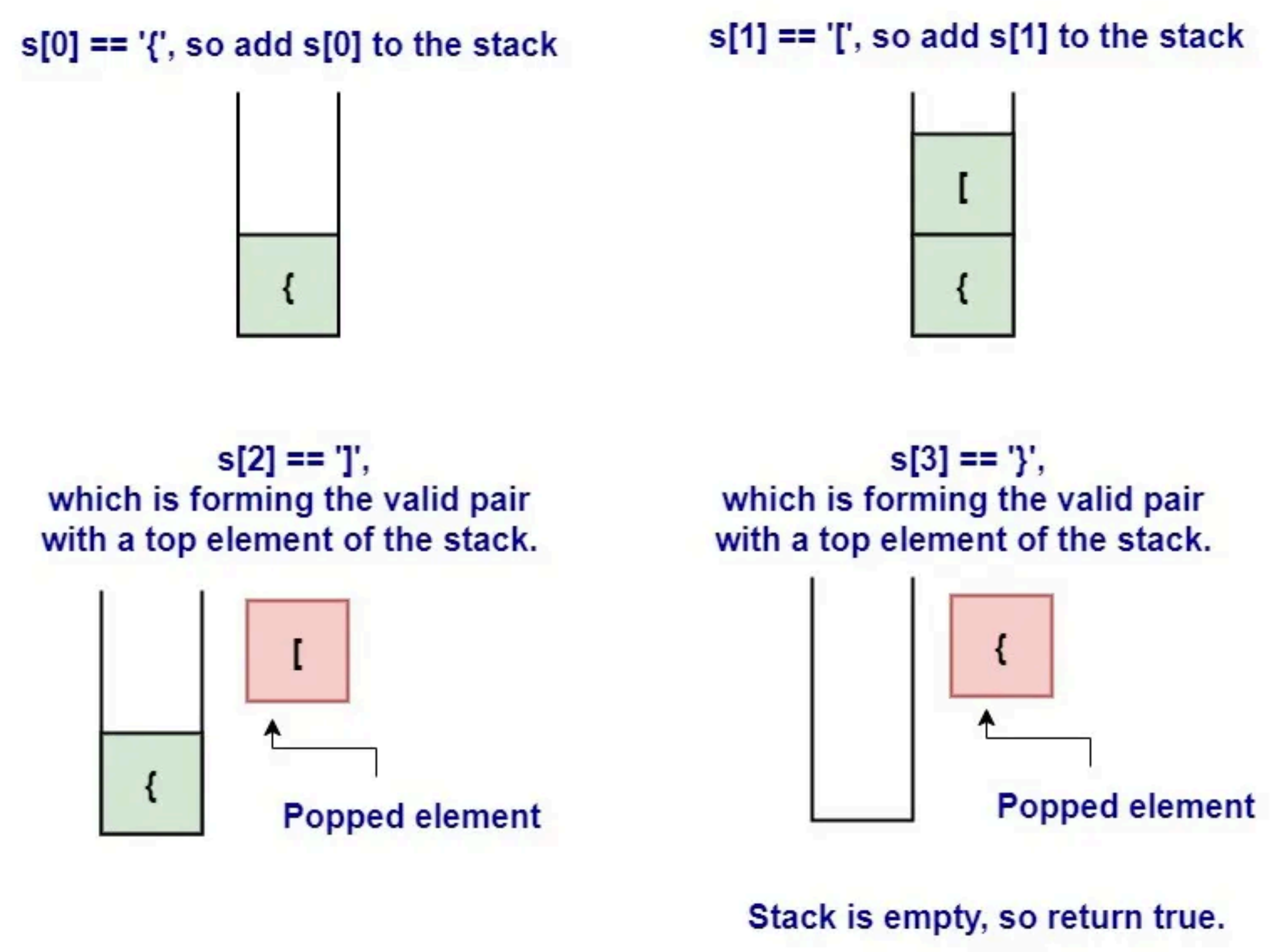
Solution

- **Initialization**:
  - Start with an empty stack.
  - For every bracket in the input string:
    - If the bracket is an opening bracket ('(', '{', '['), push it onto the stack.
    - If the bracket is a closing bracket (')', '}', ']'):
      - If the stack is empty, return false.
      - Pop the top of the stack and check if it matches the corresponding opening bracket.
- **Validation**:
  - If the current closing bracket doesn't match the top of the stack, return false.
  - If the stack is not empty after processing all brackets in the string, return false.
- **Final Output**:
  - If the stack is empty at the end, return true, indicating that the string has valid parentheses.

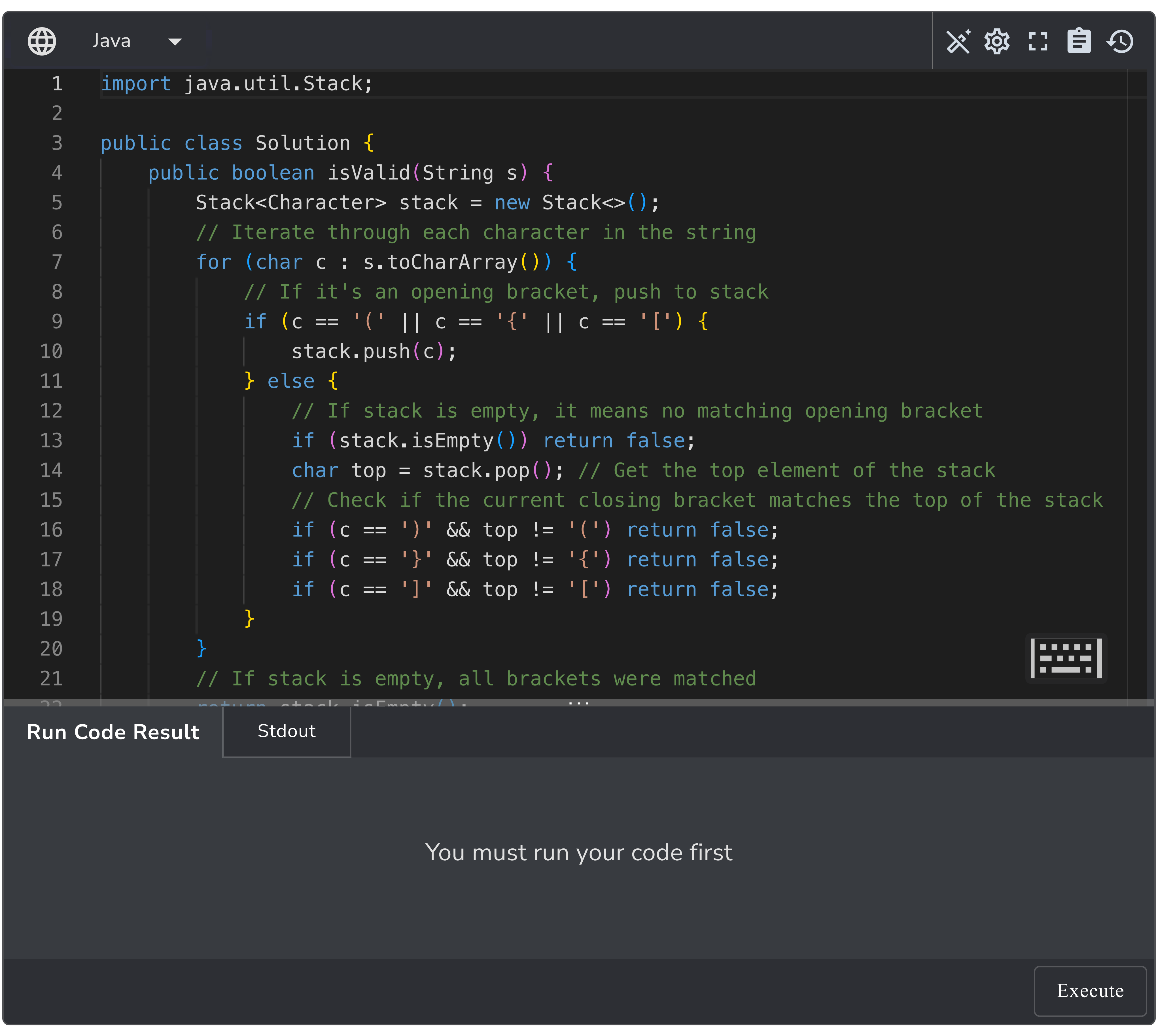
Algorithm Walkthrough

Using the input `"{[]}"`:

- Start with an empty stack.
- Encounter '{', push onto the stack.
- Encounter '[', push onto the stack.
- Encounter ']', pop from the stack and check if the popped character is '['. It matches, so continue.
- Encounter '}', pop from the stack and check if the popped character is '{'. It matches.
- The string is processed and the stack is empty, so return true.


s[0] == '{', so add s[0] to the stack
{
s[1] == '[', so add s[1] to the stack
[
{
s[2] == ']',
which is forming the valid pair
with a top element of the stack.
{
[
Popped element
s[3] == '}',
which is forming the valid pair
with a top element of the stack.
{
Popped element
Stack is empty, so return true.

Code



```java
import java.util.Stack;

public class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        // Iterate through each character in the string
        for (char c : s.toCharArray()) {
            // If it's an opening bracket, push to stack
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                // If stack is empty, it means no matching opening bracket
                if (stack.isEmpty()) return false;
                char top = stack.pop(); // Get the top element of the stack
                // Check if the current closing bracket matches the top of the stack
                if (c == ')' && top != '(') return false;
                if (c == '}' && top != '{') return false;
                if (c == ']' && top != '[') return false;
            }
        }
        // If stack is empty, all brackets were matched
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n) where n is the length of the string. Each character is processed once.
- **Space Complexity**: O(n) in the worst case when all characters are opening brackets.

← Previous
Valid Parentheses (Easy)

Next →
Subtree Of Another Tree (Easy)

☐ Mark as Completed